Thesis for Master's Degree

State-wise Safety in Autonomous Driving via Lagrangian-based Constrained Reinforcement Learning

Minseok Seo

Artificial Intelligence Graduate School

Gwangju Institute of Science and Technology

2025

석사학위논문

라그랑지안 기반 제약 강화학습을 이용한 자율주행에서의 상태별 안전성 고려

서민석

AI대학원(학과)

광주과학기술원

2025

State-wise Safety in Autonomous Driving via Lagrangian-based Constrained Reinforcement Learning

Advisor: Ue-Hwan Kim

by

Minseok Seo

Artificial Intelligence Graduate School

Gwangju Institute of Science and Technology

A thesis submitted to the faculty of the Gwangju Institute of Science and Technology in partial fulfillment of the requirements for the degree of Master of Science in the Artificial Intelligence Graduate School

Gwangju, Republic of Korea

June 20, 2025

Approved by

Professor Ue-Hwan Kim

Committee Chair

State-wise Safety in Autonomous Driving via Lagrangian-based Constrained Reinforcement Learning

Minseok Seo

Accepted in partial fulfillment of the requirements for

the degree of Master of Science

June 20, 2025

Committee Chair _____

Prof. Ue-Hwan Kim.

Committee Member _____

Prof. Kyung Joong Kim.

Committee Member _____

Prof. Kyunghwan Choi.

Dedicated to my family.

MS/AI 20231115 Minseok Seo. State-wise Safety in Autonomous Driving via Lagrangian-based Constrained Reinforcement Learning. Artificial Intelligence Graduate School. 2025. 19p. Advisor: Prof. Ue-Hwan Kim.

Abstract

For the practical deployment of autonomous driving systems, high levels of safety and adaptability are essential. Accordingly, Deep Reinforcement Learning (DRL), which learns and improves driving strategies through trial and error, has gained attention. However, the reward-driven nature of reinforcement learning may still lead to unsafe or abnormal behavior even after training. To address this limitation, Constrained Reinforcement Learning (CRL) has been proposed to balance safety and performance. While CRL typically defines constraints as expected cumulative costs, this formulation does not consider whether constraints are satisfied at each state, making it difficult to ensure state-wise safety. In this paper, we extend a Lagrangian-based CRL approach by estimating state-wise Lagrangian multipliers, allowing the policy to account for state-level safety. We evaluate the proposed method in OpenAI's Safety Gym environment and compare its performance with existing Lagrangian-based methods.

©2025
Minseok Seo
ALL RIGHTS RESERVED

MS/AI 20231115 서민석. 라그랑지안 기반 제약 강화학습을 이용한 자율주행에서의 상태별 안전성 고려. AI대학원(학과). 2025. 19p. 지도교수: 김의환.

국 문 요 약

자율주행 시스템의 실제 적용을 위해서는 높은 안정성과 적응성이 요구된다. 이에 따라, 시행착오를 통해 주행 전략을 학습하며 발전시키는 심층 강화 학습(Deep Reinforcement Learning, DRL)이 주목받고 있다. 하지만 강화 학습은 본질적으로 보상을 극대화하는 방향으로 정책을 학습하기 때문에, 학습 후에도 안전하지 않거나 비정상적인 행동을 할 가능성을 완전히 배제하기 어렵다. 이러한 한계를 해결하기 위해, 정책 학습 시 안정성과 성능 간의 균형을 도모하는 제약 강화 학습(Constrained Reinforcement Learning, CRL)이 제안되었다. 제약 강화 학습은 기댓값 기반 누적 비용 형태의 제약 조건을 만족하도록 정책을 학습하지만, 각 상태에서의 제약 조건 충족 여부를 고려하지 않아 상태별 안정성을 보장하기 어렵다. 본 논문에서는 제약 강화 학습의 한 방식인 라그랑지안 기반의 방법을 확장하여, 상태별 라그랑주 승수를 추정함으로써 정책이 상태별 안정성을 고려하도록 한다. 또한 제안한 방법을 openAI의 시뮬레이션 환경인 Safety Gym을 통해 기존 라그랑지안 기반의 방법들과 비교하여 검증하였다.

ⓒ2025
서 민 석
ALL RIGHTS RESERVED

Contents

Abstract (English) **i**

Abstract (Korean) **ii**

List of Contents **iii**

List of Tables **v**

List of Figures **vi**

List of Algorithms **vii**

1 Introduction **1**

1.1 Introduction 1

1.2 Penalized Reward Function 1

1.3 Constrained Reinforcement Learning 2

1.4 State-wise Constrained Reinforcement Learning 3

1.5 Research Objective 3

1.6 Outline of the Thesis 4

2 Background **5**

2.1 Reinforcement Learning 5

2.1.1 Policy Gradient Methods 6

2.1.2 Proximal Policy Optimization (PPO) 9

2.2 Constrained Reinforcement Learning 11

2.2.1 Lagrangian Method 11

2.2.2 PPO in Constrained Reinforcement Learning 12

2.3 State-wise Constrained Reinforcement Learning 13

2.3.1 PPO in State-wise Constrained Reinforcement Learning 15

3 PPO Lagrangian Network **17**

Summary **18**

References **19**

A Abbreviations **20**

B More appendix **21**

Acknowledgements **22**

List of Tables

List of Figures

List of Algorithms

1 PPO 10

2 PPO in Constrained Reinforcement Learning 14

3 PPO in State-wise Constrained Reinforcement Learning 16

Chapter 1

Introduction

1.1 Introduction

Reinforcement Learning (RL) is a method for learning an optimal policy through trial and error, and it has recently been applied in a wide range of domains. The goal of RL is to learn a policy that maximizes the cumulative reward, which is iteratively improved through interaction with the environment. Therefore, RL agents are typically trained in a simulated environment before deployment in the real world. However, even after training, it is difficult to fully guarantee that risky behavior does not occur. This is due to factors such as environmental uncertainty, poorly designed reward functions that fail to encourage safe behavior, or an inability to properly handle exceptional situations. This issue is especially critical in domains where reliable and safe decision making is crucial, such as robotics, autonomous driving, and healthcare. To address the potential for risky behavior, various approaches to intergrating safety into RL have been proposed.

1.2 Penalized Reward Function

One of the most straightforward approaches to ensuring safety in RL is to penalize risky behavior by modifying the reward function. This can be done by adding a penalty

term to the reward function that discourages unsafe actions. [1]

$$\max_{\pi_\theta} \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{T} (r_t - \alpha c_t) \right] \tag{1.1}$$

where r_t denotes the reward and c_t the cost at time t, and α is a hyperparameter that determines the strength of the penalty. This approach is simple and effective, but it requires careful tuning of the penalty term to ensure that the agent learns safe behavior without sacrificing performance. Moreover, it does not provide an explicit specification of constraints, which makes it difficult to guarantee that the agent will consistently satisfy safety requirements.

1.3 Constrained Reinforcement Learning

Constrained Reinforcement Learning (CRL) extends the standard RL framework by explicit constraints to ensure that the learned policy not only maximizes rewards but also satisfies specified conditions. This framework is typically formalized as a Constrained Markov Decision Process (CMDP) [2], which is defined by a tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \mathcal{C}, \gamma \rangle$. $\mathcal{C}$ is the cost function associated with the constraint. The feasible policy set in a CMDP is given by:

$$\Pi_C = \{\pi : J_c(\pi) \leq d\} \tag{1.2}$$

where $J_c(\pi)$ is a cost-based constraint function based on expected cumulative cost and d is the constraint threshold(human-defined hyperparameter). The objective of CRL is to find an optimal policy that maximizes the expected cumulative reward while

satisfying the constraints:

$$\begin{aligned} \pi^* &= \arg\max_{\pi_\theta} J(\theta) \\ J(\theta) &= \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T} r_t\right] \text{ subject to } \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T} c_t\right] \leq d \end{aligned} \tag{1.3}$$

1.4 State-wise Constrained Reinforcement Learning

State-wise Constrained Reinforcement Learning (SCRL) is a variant of CRL that imposes constraints at the state level. CRL considers the cumulative cost over the entire trajectory, while SCRL focuses the expected cost at each time step. SCRL is formalized as a State-wise Constrained Markov Decision Process (SCMDP), it is quite similar to CMDP, but SCMDP requires the cost for each state action transition satisfies a hard constraint. The objective of SCRL is to find an optimal policy that maximizes the expected cumulative reward while satisfying the state-wise constraints:

$$\begin{aligned} \pi^* &= \arg\max_{\pi_\theta} J(\theta) \\ J(\theta) &= \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T} r_t\right] \text{ subject to } \mathbb{E}_{a\sim\pi_\theta(\cdot|s)}[c(s,a)] \leq w, \quad \forall s \in S \end{aligned} \tag{1.4}$$

1.5 Research Objective

This study focuses on CRL, specifically SCRL, which is a variant of CRL that imposes constraints at the state level. We experimentally analyze the limitations of existing method that apply the Lagrangian method to the Soft Actor-Critic (SAC) algorithm. Furthermore, we propose an extension of the PPO-Lagrangian algorithm by introducing a neural network that estimates state-wise Lagrange multipliers, and

compare its performance with that of existing methods. Finally, we demonstrate that the learned Lagrange multiplier network can be used to evaluate state-wise safety throughout the policy's operation.

1.6 Outline of the Thesis

This thesis is organized as follows. In chapter 2, provides the theoretical background necessary to understand this study. It includes the fundamentals of reinforcement learning, policy gradient methods and the Lagrangian method, which are essential for understanding the proposed method.

Chapter 2

Background

2.1 Reinforcement Learning

Reinforcement Learning (RL) is a framework in which an agent interacts with an environment and learns a policy to maximize cumulative rewards. The agent observes the state of the environment and takes actions based on its policy, receiving rewards as feedback. This process is formalized as a Markov Decision Process (MDP), which provides a formal structure for decision making in RL. An MDP is defined by a tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$, where $\mathcal{S}$ is the set of states, $\mathcal{A}$ is the set of actions, $\mathcal{P}$ is the state transition probability function, $\mathcal{R}$ is the reward function, and γ is the discount factor. The objective of RL is to find an optimal policy π^* that maximizes the expected cumulative reward, defined as:

$$\begin{aligned} \theta^* &= \arg\max J(\theta) \\ J(\theta) &= \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{T} r_t \right] \end{aligned} \tag{2.1}$$

The policy is assumed to be parameterized by θ and differentiable.

2.1.1 Policy Gradient Methods

Since the policy is differentiable, its gradient can be expressed using the likelihood ratio trick:

$$\begin{aligned}\nabla_\theta \pi_\theta(s,a) &= \pi_\theta(s,a)\frac{\nabla_\theta \pi_\theta(s,a)}{\pi_\theta(s,a)} \\ &= \pi_\theta(s,a)\nabla_\theta \log \pi_\theta(s,a)\end{aligned} \tag{2.2}$$

This term $\nabla_\theta \log \pi_\theta(s,a)$ is referred to as the score function. Although we previously defined the objective function $J(\theta)$ as the expected cumulative reward, we now consider a simplified case to facilitate explanation: a one-step MDP, in which the agent takes an action from the initial state, receives a reward, and the episode terminates immediately. Then, the objective function can be written as (d is the initial state distribution):

$$\begin{aligned}J(\theta) &= \mathbb{E}_{\pi_\theta}[r] \\ &= \sum_{s\in\mathcal{S}} d(s) \sum_{a\in\mathcal{A}} \pi_\theta(s,a) R_{s,a}\end{aligned} \tag{2.3}$$

The gradient of the objective function can be computed as:

$$\begin{aligned}\nabla_\theta J(\theta) &= \sum_{s\in\mathcal{S}} d(s) \sum_{a\in\mathcal{A}} \nabla_\theta \pi_\theta(s,a) R_{s,a} \\ &= \sum_{s\in\mathcal{S}} d(s) \sum_{a\in\mathcal{A}} \pi_\theta(s,a) \nabla_\theta \log \pi_\theta(s,a) R_{s,a} \\ &= \mathbb{E}_{\pi_\theta}[\nabla_\theta \log \pi_\theta(s,a) r]\end{aligned} \tag{2.4}$$

The Policy Gradient Theorem generalizes this result to multi-step MDPs, where the objective function is defined as the expected cumulative reward over multiple time steps.

Theorem 2.1.1 (Policy Gradient Theorem)

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}[\nabla_\theta \log \pi_\theta(s,a) Q^{\pi_\theta}(s,a)] \tag{2.5}$$

In practice, the exact action-value function $Q^{\pi_\theta}(s,a)$ is typically unknown. Thus we use the empirical return(cumulative reward) as an unbiased estimate of the action-value function. This leads to the Monte Carlo Policy Gradient method, commonly known as REINFORCE algorithm. However, REINFORCE suffers from high variance in the gradient estimates due to its reliance on returns. A common approach to reducing variance is to use a critic that approximates the action-value function, i.e., $Q_W(s,a) \approx Q^{\pi_\theta}(s,a)$

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}[\nabla_\theta \log \pi_\theta(s,a) Q_W(s,a)] \tag{2.6}$$

This is the basic idea of Actor-Critic methods. Actor updates the policy parameters θ and a critic updates the value function parameters W. To further reduce variance, we can introduce a baseline function. Importantly, subtracting a baseline from the action-value function does not change the gradient estimate. Because the gradient of

the baseline function vanishes in expectation:

$$
\begin{aligned}
\mathbb{E}_{\pi_\theta}[\nabla_\theta \log \pi_\theta(s,a)B(s)] &= \sum_{s\in\mathcal{S}} d(s) \sum_{a\in\mathcal{A}} \pi_\theta(s,a)\nabla_\theta \log \pi_\theta(s,a)B(s) \\
&= \sum_{s\in\mathcal{S}} d(s) \sum_{a\in\mathcal{A}} \nabla_\theta \pi_\theta(s,a)B(s) \\
&= \sum_{s\in\mathcal{S}} d(s)B(s)\nabla_\theta \sum_{a\in\mathcal{A}} \pi_\theta(s,a) \\
&= 0
\end{aligned}
\tag{2.7}
$$

Therefore, subtracting a baseline function from the action-value function not only leaves the gradient estimate unchanged, but also significantly reduces its variance.

$$
\begin{aligned}
A^{\pi_\theta}(s,a) &= Q^{\pi_\theta}(s,a) - V^{\pi_\theta}((s) \\
\nabla_\theta J(\theta) &= \mathbb{E}_{\pi_\theta}[\nabla_\theta \log \pi_\theta(s,a)A^{\pi_\theta}(s,a)]
\end{aligned}
\tag{2.8}
$$

However, the exact advantage function is generally inaccessible in rewal-world settings, and as a result, both the action-value and value functions must be approximated. Since the TD error δ^{π_θ} is an unbiased estimate of the advantage function,

$$
\begin{aligned}
\delta^{\pi_\theta} &= r + \gamma V^{\pi_\theta}(s') - V^{\pi_\theta}(s) \\
\mathbb{E}_{\pi_\theta}[\delta^{\pi_\theta}|s,a] &= Q^{\pi_\theta}(s,a) - V^{\pi_\theta}(s) = A^{\pi_\theta}(s,a)
\end{aligned}
\tag{2.9}
$$

we can thus use δ^{π_θ} in place of the advantage function in the policy gradient expression:

$$
\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}[\nabla_\theta \log \pi_\theta(s,a)\delta^{\pi_\theta}] \tag{2.10}
$$

Hence, the TD error provides a practical and efficient way to estimate the advantage function using only the value function, allowing for effective policy updates. To summarize, the policy gradient has many equivalent formulations:

$$
\begin{aligned}
\nabla_\theta J(\theta) &= \mathbb{E}_{\pi_\theta}[\nabla_\theta \log \pi_\theta(s,a) v_t] && \text{REINFORCE} \\
&= \mathbb{E}_{\pi_\theta}[\nabla_\theta \log \pi_\theta(s,a) Q_W(s,a)] && \text{Q Actor-Critic} \\
&= \mathbb{E}_{\pi_\theta}[\nabla_\theta \log \pi_\theta(s,a) A^{\pi_\theta}(s,a)] && \text{Advantage Actor-Critic} \\
&= \mathbb{E}_{\pi_\theta}[\nabla_\theta \log \pi_\theta(s,a) \delta^{\pi_\theta}] && \text{TD Actor-Critic}
\end{aligned}
\tag{2.11}
$$

2.1.2 Proximal Policy Optimization (PPO)

The policy gradient in (2.10), used in TD Actor-Critic, can be modified using importance sampling as follows:

$$
\begin{aligned}
\nabla_\theta J(\theta) &= \mathbb{E}_{\pi_\theta}\left[\nabla_\theta \log \pi_\theta(s,a) A^{\pi_\theta}(s,a)\right] \\
&= \mathbb{E}_{\pi_{\theta_{\text{old}}}}\left[\frac{\pi_\theta(s,a)}{\pi_{\theta_{\text{old}}}(s,a)} \nabla_\theta \log \pi_\theta(s,a) A^{\pi_\theta}(s,a)\right] \\
&= \mathbb{E}_{\pi_{\theta_{\text{old}}}}\left[\frac{\nabla_\theta \pi_\theta(s,a)}{\pi_{\theta_{\text{old}}}(s,a)} A^{\pi_\theta}(s,a)\right] \\
&\approx \mathbb{E}_{\pi_{\theta_{\text{old}}}}\left[\frac{\nabla_\theta \pi_\theta(s,a)}{\pi_{\theta_{\text{old}}}(s,a)} A^{\pi_{\theta_{\text{old}}}}(s,a)\right]
\end{aligned}
\tag{2.12}
$$

While using TD errors help reduce the variance of the gradient estimates, they do not directly constrain how much the policy changes during an update. PPO is desinged to improve the current policy as much as possible using the collected data, while avoiding large policy updates that can lead to instability. To this end, based on the equation (2.12), PPO updates the policy by maximizing a clipped surrogate objective that con-

strains the probability ratio $r_t(\theta) = \frac{\pi_\theta(s,a)}{\pi_{\theta_{\text{old}}}(s,a)}$.

$$\nabla_\theta J^{\text{PPO}}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t \right) \right] \tag{2.13}$$

This prevents the new policy from deviating too far from the old policy, promoting stable and conservative updates.

Algorithm 1: PPO

Input: Initial policy parameters θ_0, initial value function parameters ϕ_0

for *each epoch* $k = 0, 1, 2, \cdots$ **do**

  for *each time step* $t = 1$ **to** T **do**

    `// Collect trajectories`

    Sample action $a_t \sim \pi_{\theta_{t-1}}(s_t)$

    Execute action a_t in the environment and observe reward r_t, cost c_t and next state s_{t+1}

    Store transition $\tau_t = (s_t, a_t, r_t, c_t, s_{t+1})$ in buffer B_k

    if *episode end* **then**

      Compute rewards-to-go $\hat{R}_t$ and advantage estimates $\hat{A}_t$ based on the current value function V_{ϕ_k}

  `// Policy update`

  Update the policy parameters θ by maximizing the objective function:

$$\theta_{k+1} = \arg\max \frac{1}{|B_k|T} \sum_{\tau \in B_k} \sum_{t=0}^{T} \min \left(r_t(\theta) A^{\pi_{\theta_k}(s_t,a_t)},\ g\left(\epsilon, A^{\pi_{\theta_k}}(s_t, a_t)\right) \right)$$

$$\text{where } r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_k}(a_t|s_t)},\ g(\epsilon, A) = \begin{cases} (1+\epsilon)A & A \geq 0 \\ (1-\epsilon)A & A < 0 \end{cases}$$

  `// Value function update`

  Fit value function by regression on mean-squared error:

$$\phi_{k+1} = \arg\min_\phi \frac{1}{|B_k|T} \sum_{\tau \in B_k} \sum_{t=0}^{T} \left(V_\phi(s_t) - \hat{R}_t \right)^2$$

  typically via stochastic gradient descent algorithm.

2.2 Constrained Reinforcement Learning

Constrained Reinforcement Learning (CRL) extends the standard RL framework by explicit constraints to ensure that the learned policy not only maximizes rewards but also satisfies specified conditions. This framework is typically formalized as a Constrained Markov Decision Process (CMDP), which is defined by a tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \mathcal{C}, \gamma \rangle$. $\mathcal{C}$ is the cost function associated with the constraint. The feasible policy set in a CMDP is given by:

$$\Pi_C = \{\pi : J_c(\pi) \leq d\} \tag{2.14}$$

where $J_c(\pi)$ is a cost-based constraint function based on expected cumulative cost and d is the constraint threshold(human-defined hyperparameter). The objective of CRL is to find an optimal policy that maximizes the expected cumulative reward while satisfying the constraints:

$$\begin{aligned} \pi^* &= \arg\max_{\pi_\theta} J(\theta) \\ J(\theta) &= \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{T} r_t \right] \text{ subject to } \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{T} c_t \right] \leq d \end{aligned} \tag{2.15}$$

2.2.1 Lagrangian Method

Constrained optimization problem (2.15) can be reformulated as an unconstrained optimization problem using the Lagrangian relaxation. The constraint is incorporated

into the objective function using a Lagrange multiplier λ:

$$\begin{aligned} \pi^* &= \arg\max_{\pi_\theta} \mathcal{L}(\theta, \lambda) \\ \mathcal{L}(\theta, \lambda) &= \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{T} r_t \right] - \lambda \left(\mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{T} c_t \right] - d \right) \end{aligned} \tag{2.16}$$

The Lagrange multiplier λ is a non-negative scalar ($\lambda \geq 0$) that adjusts the trade-off between maximizing the reward and satisfying the constraint.

2.2.2 PPO in Constrained Reinforcement Learning

PPO can be adapted to the CRL setting by incorporating the Lagrangian method into the policy update process. PPO updates the policy by maximizing a surrogate objective that includes the Lagrangian term:

$$\nabla_\theta J^{\text{PPO-CRL}}(\theta) = \hat{\mathbb{E}}_t \left[\nabla_\theta \left\{ \min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) - \lambda r_t(\theta) \hat{A}_t^c \right\} \right] \tag{2.17}$$

where λ is a Lagrange multiplier that adjusts the trade-off between the reward and the cost, and $\hat{A}_t^c$ denotes the cost advantage function. The Lagrange multiplier λ is updated iteratively during training to ensure the constraint is satisfied. The update of the Lagrange multiplier depends on whether the empirical cumulative cost exceeds the threshold d. Specifically, the empirical cumulative cost is computed as the average cumulative cost:

$$\hat{J}_c = \frac{1}{N} \sum_{i=1}^{n} \sum_{t=0}^{T} c_t^{(i)} \tag{2.18}$$

where n is the number of sampled episodes, and $c_t^{(i)}$ denotes the cost received at time step t in the i-th episode. The Lagrange multiplier λ is updated via gradient ascent:

$$\lambda \leftarrow \left[\lambda + \beta \left(\hat{J}_c - d\right)\right]_+ \tag{2.19}$$

where β is the leraning rate, and $[\cdot]_+$ denotes clipped to be non-negative. This update increases λ when the empirical cost exceeds the threshold d, encouraging the policy to reduce the cost in subsequent updates.

2.3 State-wise Constrained Reinforcement Learning

State-wise Constrained Reinforcement Learning (SCRL) is a variant of CRL that imposes constraints at the state level. CRL considers the cumulative cost over the entire trajectory, while SCRL focuses the expected cost at each time step. SCRL is formalized as a State-wise Constrained Markov Decision Process (SCMDP), it is quite similar to CMDP, but SCMDP requires the cost for each state action transition satisfies a hard constraint. The objective of SCRL is to find an optimal policy that maximizes the expected cumulative reward while satisfying the state-wise constraints:

$$\begin{aligned} \pi^* &= \arg\max_{\pi_\theta} J(\theta) \\ J(\theta) &= \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T} r_t\right] \text{ subject to } \mathbb{E}_{a\sim\pi_\theta(\cdot|s)}[c(s,a)] \leq w, \quad \forall s \in S \end{aligned} \tag{2.20}$$

Equation (2.20) can be reformulated as an unconstrained optimization problem using the Lagrangian relaxation. This can be achieved by introducing a Lagrange multiplier

Algorithm 2: PPO in Constrained Reinforcement Learning

Input: Initial policy parameters θ_0, initial value function parameters ϕ_0, initial cost value function parameters ϕ_0^c, Lagrange multiplier λ_0, threshold d

for *each epoch* $k = 0, 1, 2, \cdots$ **do**

  for *each time step* $t = 1$ **to** T **do**

    `// Collect trajectories`

    Sample action $a_t \sim \pi_{\theta_{t-1}}(s_t)$

    Execute action a_t in the environment and observe reward r_t, cost c_t and next state s_{t+1}

    Store transition $\tau_t = (s_t, a_t, r_t, c_t, s_{t+1})$ in buffer B_k

    if *episode end* **then**

      Compute rewards-to-go $\hat{R}_t$ and advantage estimates $\hat{A}_t$ based on the current value function V_{ϕ_k}

      Compute costs-to-go $\hat{R}_t^c$ and cost advantage estimates $\hat{A}_t^c$ based on the current cost value function $V_{\phi_k}^c$

  `// Lagrange multiplier update`

  Update the Lagrange multiplier λ by gradient ascent:

$$\hat{J}_c = \frac{1}{|B_k|T} \sum_{\tau \in B_k} \sum_{t=0}^{T} c_t$$

$$\lambda_{k+1} = \left[\lambda_k + \beta \left(\hat{J}_c - d\right)\right]_+$$

  `// Policy update`

  Update the policy parameters θ by maximizing the objective function:

$$\theta_{k+1} = \arg\max \frac{1}{|B_k|T} \sum_{\tau \in B_k} \sum_{t=0}^{T} \min\left(r_t(\theta) A^{\pi_{\theta_k}(s_t,a_t)},\ g\left(\epsilon, A^{\pi_{\theta_k}}(s_t, a_t)\right)\right) - \lambda_{k+1} r_t(\theta) \hat{A}_t^{c,\pi_{\theta_k}(s_t,a_t)}$$

$$\text{where } r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_k}(a_t|s_t)},\ g(\epsilon, A) = \begin{cases} (1+\epsilon)A & A \geq 0 \\ (1-\epsilon)A & A < 0 \end{cases}$$

  `// Value function update`

  Fit value function by regression on mean-squared error:

$$\phi_{k+1} = \arg\min_{\phi} \frac{1}{|B_k|T} \sum_{\tau \in B_k} \sum_{t=0}^{T} \left(V_\phi(s_t) - \hat{R}_t\right)^2$$

$$\phi_{k+1}^c = \arg\min_{\phi} \frac{1}{|B_k|T} \sum_{\tau \in B_k} \sum_{t=0}^{T} \left(V_{\phi^c}(s_t) - \hat{R}_t^c\right)^2$$

  typically via stochastic gradient descent algorithm.

$\lambda(s) \geq 0$ for each state to penalize violations of the state-wise constraints.

$$\mathcal{L}(\theta, \lambda) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{T} r_t \right] - \sum_{s \in \mathcal{S}} \lambda(s) \left(\mathbb{E}_{a \sim \pi_\theta(\cdot|s)}[c(s,a)] - w \right) \tag{2.21}$$

2.3.1 PPO in State-wise Constrained Reinforcement Learning

PPO can also be extended to the SCRL setting by incorporating the state-wise constraints into the policy update process.

$$\nabla_\theta J^{\text{PPO-SCRL}}(\theta) = \hat{\mathbb{E}}_t \left[\nabla_\theta \left\{ \min \left(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t \right) - \lambda(s_t) r_t(\theta) \hat{A}_t^c \right\} \right] \tag{2.22}$$

where $\lambda(s_t)$ is the Lagrange multiplier for the state s_t at time step t, and $\hat{A}_t^c$ denotes the cost advantage function. The Lagrange multiplier $\lambda(s)$ is updated iteratively during training to ensure the state-wise constraint is satisfied. The update of the Lagrange multiplier $\lambda(s)$ depends on whether the empirical cost at state s exceeds the threshold w. Specifically, the empirical cost refers to the actual cost received by the agent at each time step during interaction with the environment.

$$\lambda(s) \leftarrow \lambda(s) + \beta(\hat{J}_c - w) \tag{2.23}$$

Algorithm 3: PPO in State-wise Constrained Reinforcement Learning

Input: Initial policy parameters θ_0, initial value function parameters ϕ_0, initial cost value function parameters ϕ_0^c, Lagrange multiplier network parameters ψ, threshold w

for *each epoch* $k = 0, 1, 2, \cdots$ **do**

  for *each time step* $t = 1$ **to** T **do**

    `// Collect trajectories`

    Sample action $a_t \sim \pi_{\theta_{t-1}}(s_t)$

    Execute action a_t in the environment and observe reward r_t, cost c_t and next state s_{t+1}

    Store transition $\tau_t = (s_t, a_t, r_t, c_t, s_{t+1})$ in buffer B_k

    if *episode end* **then**

      Compute rewards-to-go $\hat{R}_t$ and advantage estimates $\hat{A}_t$ based on the current value function V_{ϕ_k}

      Compute costs-to-go $\hat{R}_t^c$ and cost advantage estimates $\hat{A}_t^c$ based on the current cost value function $V_{\phi_k}^c$

  `// Lagrange multiplier network update`

  Update the Lagrange multiplier λ by gradient ascent:

$$\psi_{k+1} = \psi_k + \beta \left[\frac{1}{|B_k|T} \sum_{\tau \in B_k} \sum_{t=0}^{T} (c_t - w) \right]$$

  `// Policy update`

  Update the policy parameters θ by maximizing the objective function:

$$\theta_{k+1} = \arg\max \frac{1}{|B_k|T} \sum_{\tau \in B_k} \sum_{t=0}^{T} \min \left(r_t(\theta) A^{\pi_{\theta_k}(s_t, a_t)},\ g\left(\epsilon, A^{\pi_{\theta_k}}(s_t, a_t)\right) \right) - \lambda_\psi(s_t) r_t(\theta) \hat{A}_t^{c, \pi_{\theta_k}(s_t, a_t)}$$

$$\text{where } r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_k}(a_t|s_t)},\ g(\epsilon, A) = \begin{cases} (1+\epsilon)A & A \geq 0 \\ (1-\epsilon)A & A < 0 \end{cases}$$

  `// Value function update`

  Fit value function by regression on mean-squared error:

$$\phi_{k+1} = \arg\min_{\phi} \frac{1}{|B_k|T} \sum_{\tau \in B_k} \sum_{t=0}^{T} \left(V_\phi(s_t) - \hat{R}_t \right)^2$$

$$\phi_{k+1}^c = \arg\min_{\phi} \frac{1}{|B_k|T} \sum_{\tau \in B_k} \sum_{t=0}^{T} \left(V_{\phi^c}(s_t) - \hat{R}_t^c \right)^2$$

  typically via stochastic gradient descent algorithm.

Chapter 3

PPO Lagrangian Network

Summary

State-wise Safety in Autonomous Driving

via Lagrangian-based

Constrained Reinforcement Learning

Summarize your thesis.

References

1. C. Tessler, D. J. Mankowitz, and S. Mannor, "Reward constrained policy optimization," *arXiv preprint arXiv:1805.11074*, 2018.

2. E. Altman, *Constrained Markov decision processes.* Routledge, 2021.

Appendix A

Abbreviations

GIST Gwangju Institute of Science and Technology

EECS Electrical Engineering and Computer Science

YOLO You Only Live Once

Appendix B

More appendix

You can add more appendices.

Acknowledgements

special thanks to...

모두들 감사합니다.